CREATE HASH VALUES

PORTFOLIO ACTIVITY · HASHING & FILE INTEGRITY

ANALYST	ROLE	DATE	REFERENCE
Kelly Guzman	**Founder & SOC Analyst, CTA**	**May 13, 2026**	**Coursera · Course 3**

Overview

This activity used Linux command-line tools to generate cryptographic hash values for two files containing the EICAR antivirus test string. Both files appear identical when viewed with `cat`, but hashing reveals byte-level differences that visual inspection alone cannot detect. This is the same workflow used in malware analysis, evidence integrity verification, and file tamper detection.

File contents

Both files contained the EICAR Standard Antivirus Test File string. EICAR is a safe, industry-standard string designed to test antivirus detection without using actual malware.

```
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
```

Although the contents of both files appeared identical when viewed with the `cat` command, hashing each file was needed to determine whether the files were actually different.

```
analyst@e204d7b3034d:~$ ls
file1.txt  file2.txt
analyst@e204d7b3034d:~$ cat file1.txt
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
analyst@e204d7b3034d:~$ cat file2.txt
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
9sxa5Yq20Ranalyst@e204d7b3034d:~$
```

Generate hashes

Used `sha256sum` to generate SHA-256 hashes for both files. The two outputs differed completely, confirming the files were not byte-identical despite the visual match.

131f95c51cc819465fa1797f6ccacf9d494aaaff46fa3eac73ae63ffbdfd8267 file1.txt

2558ba9a4cad1e69804ce03aa2a029526179a91a5e38cb723320e83af9ca017b file2.txt

```
analyst@e204d7b3034d:~$ cmp file1hash file2hash
file1hash file2hash differ: char 1, line 1
```

```
analyst@e204d7b3034d:~$ sha256sum file1.txt
131f95c51cc819465fa1797f6ccacf9d494aaaff46fa3eac73ae63ffbdfd8267  file1.txt
```

Write hashes to separate files

Each hash was written to its own file for later comparison. The first hash was written to `file1hash` and the second to `file2hash`.

```
analyst@e204d7b3034d:~$ sha256sum file2.txt
2558ba9a4cad1e69804ce03aa2a029526179a91a5e38cb723320e83af9ca017b  file2.txt
```

```
analyst@e204d7b3034d:~$ cat file1hash
131f95c51cc819465fa1797f6ccacf9d494aaaff46fa3eac73ae63ffbdfd8267  file1.txt
analyst@e204d7b3034d:~$ cat file2hash
2558ba9a4cad1e69804ce03aa2a029526179a91a5e38cb723320e83af9ca017b  file2.txt
```

Compare files byte by byte

Used the `cmp` command to compare the two hash files byte by byte. If a difference is found, the command reports the byte and line number where the first difference appears.

```
analyst@e204d7b3034d:~$ ls
file1.txt  file2.txt
analyst@e204d7b3034d:~$ cat file1.txt
X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
analyst@e204d7b3034d:~$
```

Summary

Two files with visually identical contents produced completely different SHA-256 hashes, confirming a byte-level difference invisible to `cat`. This demonstrates the core security principle behind hashing: any change to a file, no matter how small, produces a completely different hash output. Hashing is the foundation for file integrity verification, malware identification, evidence preservation in incident response, and tamper detection in critical systems.